

Problem Definition

My client, Mr. L, is a heavy cashless payment user who is seeking a ledger that allows him to track their transactions on various (credit/debit) cards. Currently, L is manually inputting spending history into a ledger application. However, L has come across several issues with the ledger app he currently uses.

The application he currently uses doesn't take into account which card was used, making it difficult to understand his spendings fully aside from the total amount across all cards. To find an overall summary of expenditures for each card, L must look at each app associated with the cards to total individual transactions, separate from the ledger he uses to track the overall expenditures.

He also wishes to plan ahead in time, by checking the expected cashback/points from each of his spendings. However, as each card has its own rates, he must look at individual apps and calculate himself, or wait until the cashback each month to see the actual amount.

Furthermore, L travels and invests frequently, therefore he owns various currencies. This means that when he makes a transaction in a different currency, he must manually convert it back into a single currency to enter in the ledger application.

To tackle these issues, Mr. L is looking for a ledger application where he can input transaction data so details such as the card that was used, the transaction category, and currency used are considered.

(See Appendix A for evidence)

Proposed Solution

Using Swift, SwiftUI, and SwiftData, I will create an iOS application for an iPhone in which a user can track expenditures and store the data to look back at later.

As L is mainly an iPhone user, an application accessible on his phone for daily use, must be an iOS-compatible application. Hence, there is a need to develop the solution using the Swift language. Swift is a compiled language developed by Apple that is most commonly used to develop applications for devices in the Apple ecosystem. Software updates from Apple have provided support for Swift to be run on all modern Apple devices, reducing the need for the client to download another language such as Python on the device intended to run the application. (Apple)

Furthermore, SwiftUI is a declarative framework for the Swift language which allows for the building of user interfaces for applications on Apple devices. SwiftUI is available on all iPhone

devices using iOS 13+, reducing the need for the user to download massive amounts of data with the installation of the app. (Apple)

Success Criteria

1. The application allows users to create different cards under their account, assigning details such as name, default currency, cashback rate, etc.
2. The application allows users to input customized expenses, being able to select the date, card used, the category of the transaction, and the currency used.
3. The application allows for expenses to be deleted and edited.
4. The application can display cumulative statistics (e.g. total expenditures, total expected cashback) of expenses by card, and by month.
5. The application can display pie charts of a breakdown of expenses by category from cumulative and per-card monthly expenditures.
6. If an expense is made in a non-default currency, the application obtains the exchange rate between two given currencies on the day of transaction to convert the currency.

Works Cited

Apple. "Swift." *Apple*, 2024, <https://www.apple.com/lae/swift/>. Accessed 2 September 2024.

Apple. "Swift." *Apple Developer*, 2024, <https://developer.apple.com/swift/>. Accessed 2 September 2024.

Apple. "SwiftUI." *Apple Developer*, 2024, <https://developer.apple.com/documentation/swiftui/>. Accessed 2 September 2024.

Parmar, Ashish. "8 Key Reasons Why using Swift for iOS app development is a Perfect decision." *Prismetric*, 25 August 2023, <https://www.prismetric.com/using-swift-for-ios-app-development/>. Accessed 2 September 2024.

(418 words)